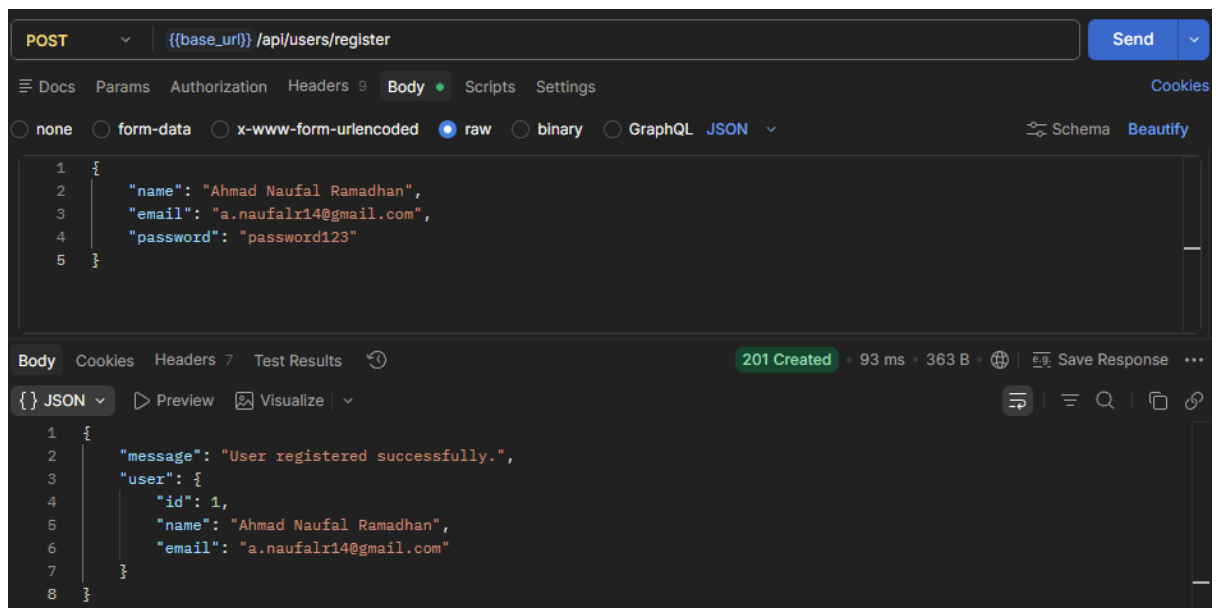


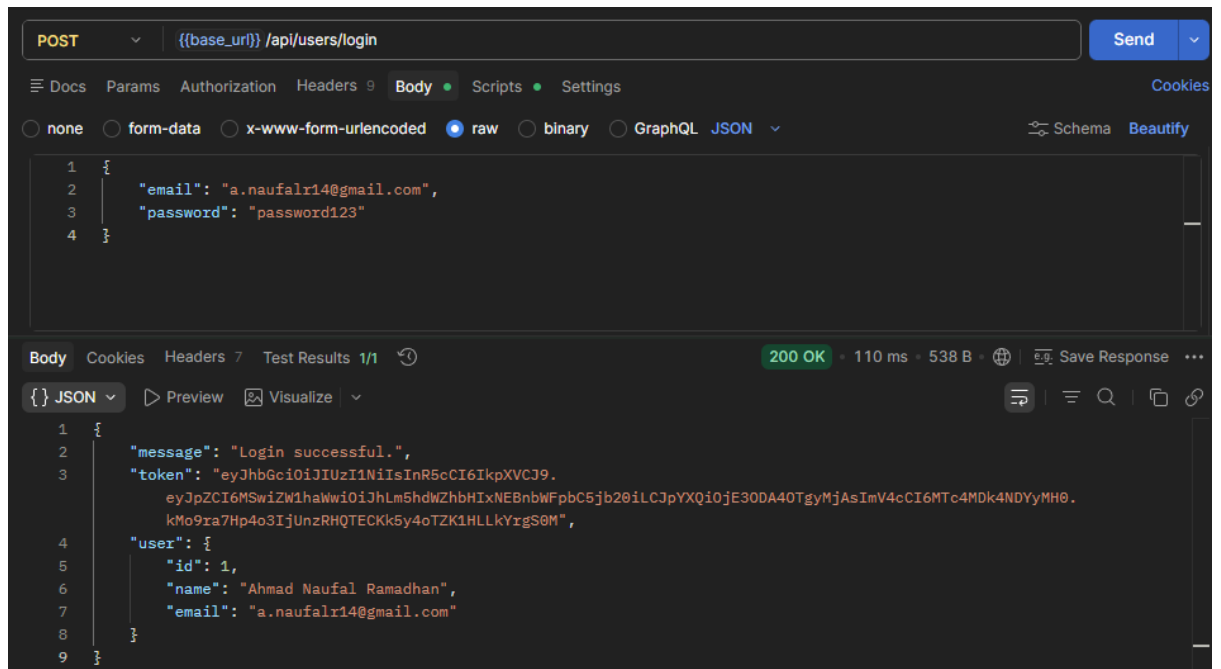
- Register



Pengujian ini bertujuan untuk memvalidasi fungsionalitas endpoint registrasi pengguna baru pada sistem. Request dikirimkan menggunakan metode HTTP POST dengan format body bertipe raw JSON, yang memuat tiga parameter wajib, yakni name, email, dan password.

Setelah request berhasil diproses, server mengembalikan respons dengan kode status 201 Created, yang secara semantik menandakan bahwa resource baru telah berhasil dibuat dan disimpan ke dalam basis data. Data yang dikembalikan dalam respons meliputi id, nama, serta alamat email pengguna yang baru terdaftar. Perlu dicatat bahwa field password tidak disertakan dalam respons, hal tersebut mencerminkan penerapan prinsip keamanan data yang sesuai dengan standar pengembangan perangkat lunak yang baik.

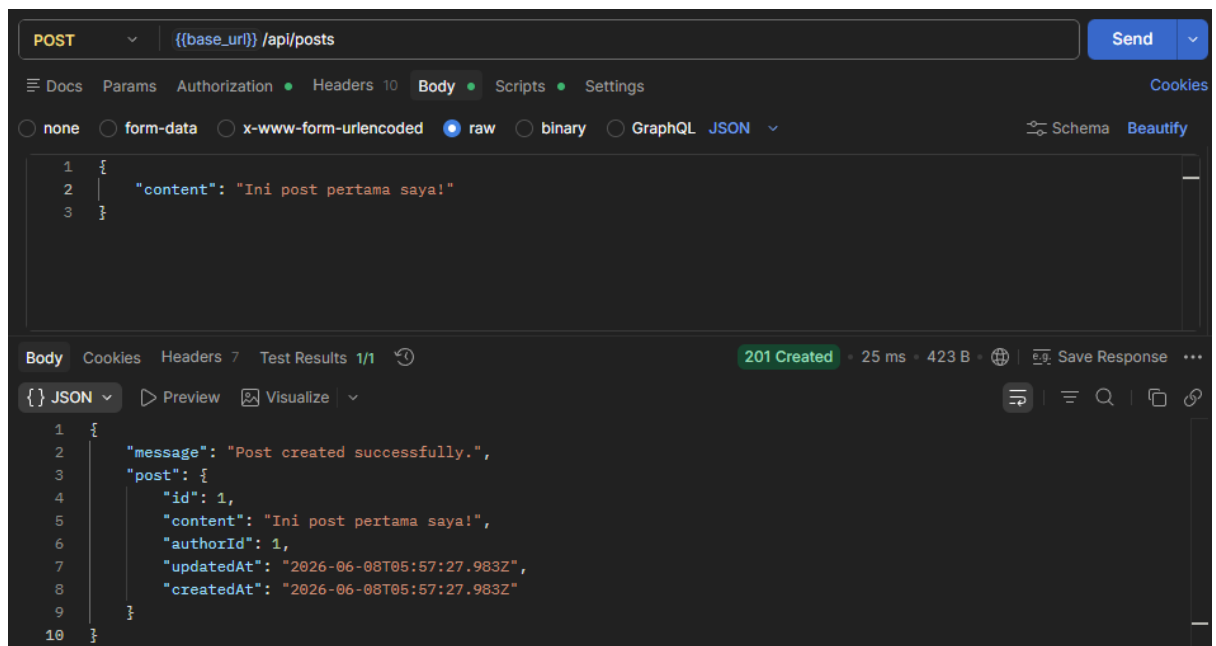
- Login



Endpoint ini dirancang untuk menangani proses autentikasi pengguna yang telah terdaftar sebelumnya dalam sistem. Request dikirimkan dengan menyertakan dua parameter pada body, yaitu email dan password sebagai kredensial pengguna.

Apabila proses verifikasi kredensial berjalan dengan sukses, server memberikan respons dengan kode status 200 OK beserta JSON Web Token (JWT) yang berfungsi sebagai token autentikasi. Token tersebut merupakan komponen krusial yang wajib disertakan pada setiap request berikutnya yang memerlukan otorisasi, guna memastikan bahwa hanya pengguna yang sah yang dapat mengakses sumber daya terproteksi pada sistem.

- POST api posts



Endpoint ini digunakan untuk membuat entitas postingan baru di dalam sistem. Berdasarkan hasil pengujian, terlihat bahwa tab Authorization menampilkan indikator aktif, yang mengonfirmasi bahwa JWT token yang diperoleh dari proses autentikasi sebelumnya wajib disertakan sebagai bagian dari header request. Hal ini menunjukkan bahwa endpoint tersebut bersifat terproteksi dan hanya dapat diakses oleh pengguna yang telah melalui proses autentikasi.

Server merespons dengan kode status 201 Created dan mengembalikan data postingan yang baru dibuat, mencakup id, content, authorId, serta informasi timestamp dalam format ISO 8601 untuk field createdAt dan updatedAt.

- GET api posts

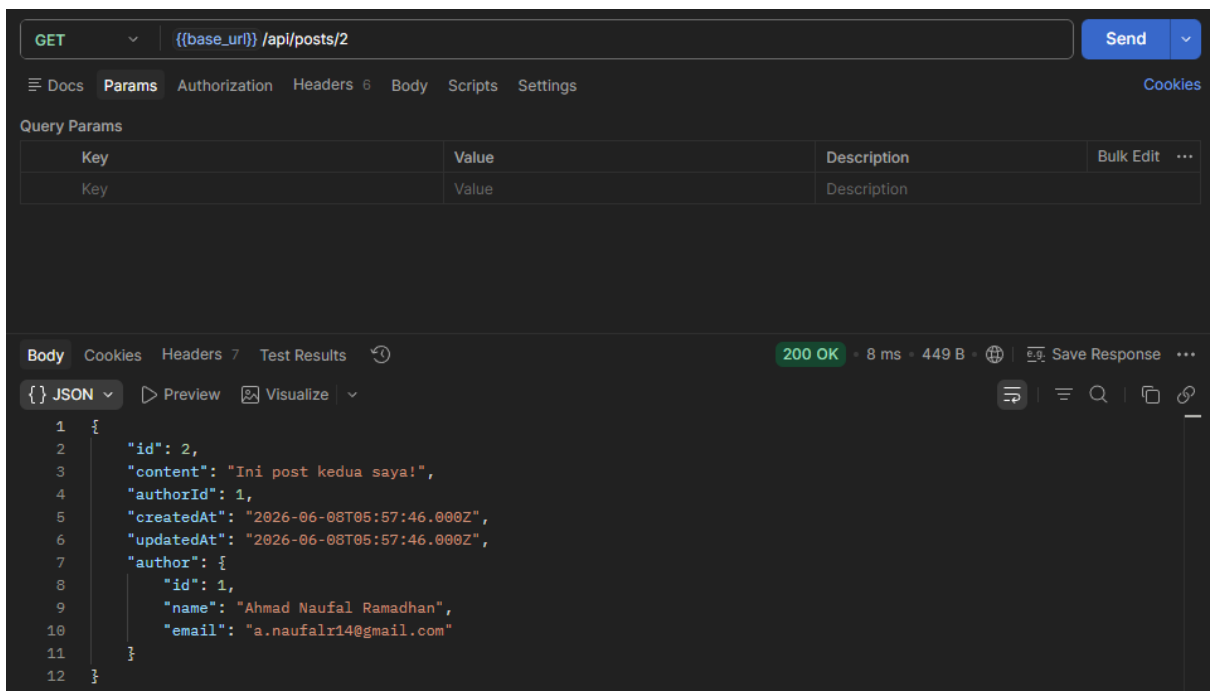
The screenshot shows a REST client interface with the following details:

- Method:** GET
- URL:** {{base_url}} /api/posts
- Status:** 200 OK
- Response Time:** 14 ms
- Response Size:** 883 B
- Body:** JSON array of two posts.

```
1 [
2   {
3     "id": 3,
4     "content": "Ini post ketiga saya!",
5     "authorId": 1,
6     "createdAt": "2026-06-08T05:57:52.000Z",
7     "updatedAt": "2026-06-08T05:57:52.000Z",
8     "author": {
9       "id": 1,
10      "name": "Ahmad Naufal Ramadhan",
11      "email": "a.naufalr14@gmail.com"
12    }
13  },
14  {
15    "id": 2,
16    "content": "Ini post kedua saya!",
17    "authorId": 1,
18    "createdAt": "2026-06-08T05:57:46.000Z",
19    "updatedAt": "2026-06-08T05:57:46.000Z",
```

Endpoint ini digunakan untuk mengambil keseluruhan data postingan yang tersimpan dalam sistem. Server mengembalikan respons dengan kode status 200 OK dalam bentuk array JSON yang memuat seluruh objek postingan yang ada. Hal yang patut diperhatikan adalah data author telah disertakan secara langsung di dalam setiap objek postingan melalui mekanisme relasi antar tabel, sehingga informasi mengenai penulis dapat diperoleh tanpa memerlukan request tambahan ke endpoint terpisah.

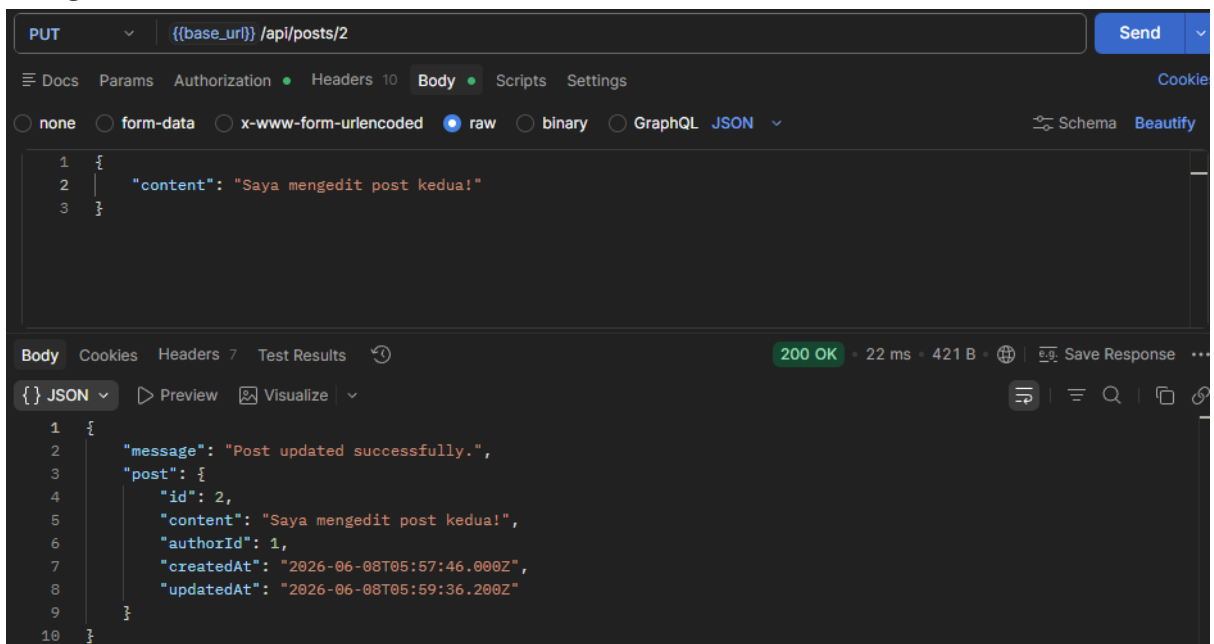
- GET api posts berdasarkan id



Endpoint ini berfungsi untuk mengambil satu data postingan secara spesifik berdasarkan parameter ID yang disisipkan sebagai bagian dari URL path.

Respons yang dikembalikan memiliki struktur yang identik dengan endpoint pengambilan semua postingan, namun hanya memuat satu objek tunggal alih-alih array. Data relasi author tetap disertakan di dalamnya. Response time yang tercatat sebesar 20ms mengindikasikan performa endpoint yang memadai.

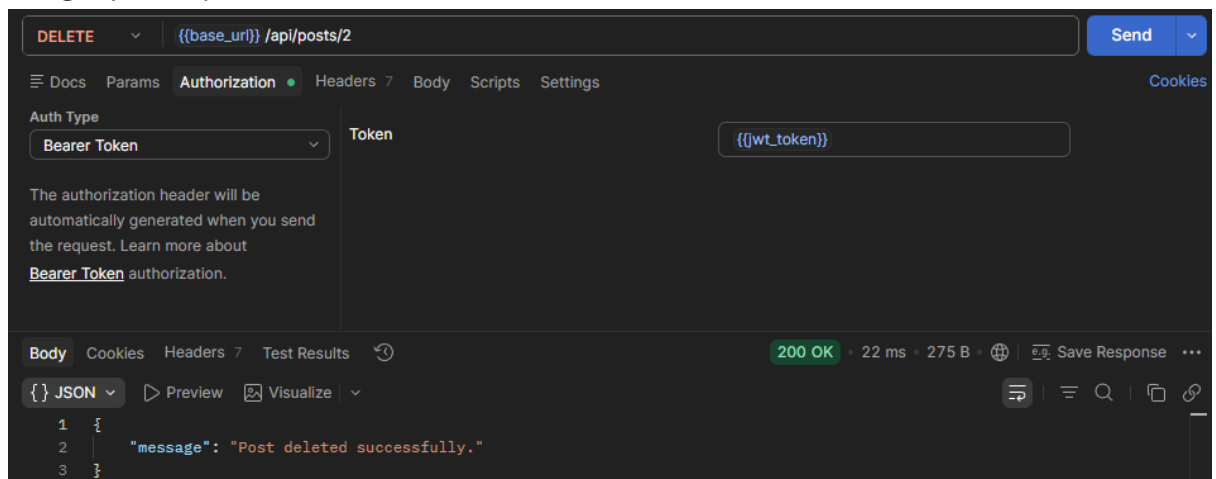
- Mengedit konten



Endpoint ini digunakan untuk melakukan pembaruan terhadap konten postingan yang telah ada berdasarkan ID yang ditentukan. Tab Authorization kembali menampilkan status aktif pada request ini, yang secara logis diperlukan guna

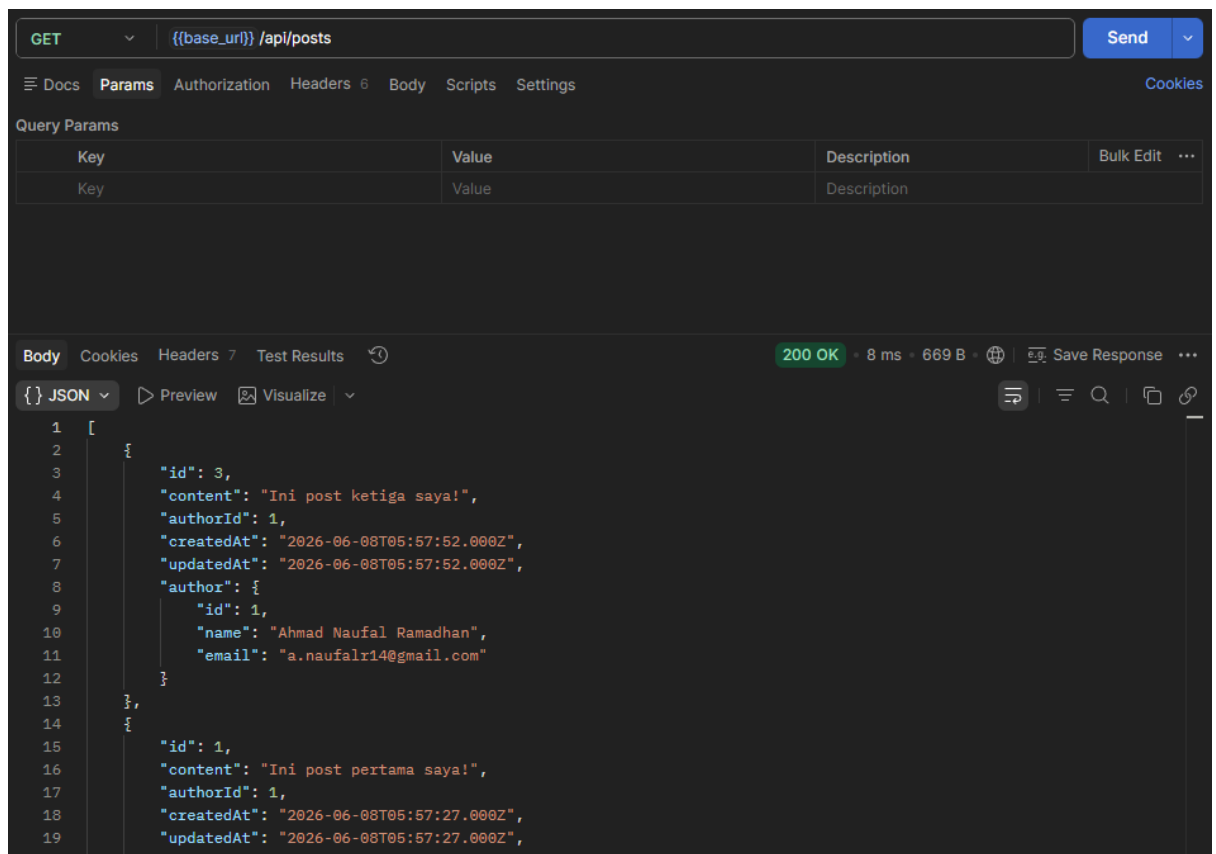
memastikan bahwa hanya pengguna yang memiliki hak akses terhadap postingan tersebut yang dapat melakukan modifikasi data. Server merespons dengan kode status 200 OK disertai pesan konfirmasi keberhasilan serta data postingan yang telah diperbarui. Terdapat temuan yang perlu mendapat perhatian lebih lanjut, yaitu nilai pada field `updatedAt` menampilkan waktu yang secara kronologis lebih awal dibandingkan nilai pada field `createdAt`. Kondisi ini mengindikasikan adanya potensi inkonsistensi data atau kesalahan logika pada lapisan backend yang perlu ditelusuri dan diperbaiki.

- Penghapusan post



Endpoint ini digunakan untuk menghapus data postingan secara permanen dari sistem berdasarkan ID yang diberikan melalui URL path. Sebagaimana endpoint sebelumnya yang bersifat terproteksi, Authorization tetap aktif pada request ini guna memastikan integritas dan keamanan proses penghapusan data. Server mengembalikan respons yang ringkas berupa kode status 200 OK disertai pesan konfirmasi singkat yang menyatakan bahwa proses penghapusan telah berhasil dilaksanakan.

- Verifikasi setelah dilakukan penghapusan



Sebagai langkah verifikasi akhir, request GET untuk mengambil seluruh data postingan dieksekusi kembali setelah proses penghapusan dilakukan. Hasil yang dikembalikan berupa array kosong `[]`, yang secara konklusif membuktikan bahwa operasi penghapusan telah berjalan sebagaimana mestinya dan tidak terdapat data residual yang tersisa dalam sistem. Dengan demikian, keseluruhan siklus pengujian operasi CRUD pada rangkaian endpoint ini dinyatakan telah selesai dilaksanakan secara menyeluruh dan berhasil.

- Unit Test

```
C:\xampp\htdocs\Test Backend Engineer>npm test

> test-backend-engineer@1.0.0 test
> jest --runInBand --detectOpenHandles --forceExit

console.log
  ◊ injected env (7) from .env // tip: % custom filepath { path: '/custom/path/.env' }

    at _log (node_modules/dotenv/lib/main.js:131:11)

console.log
  ◊ injected env (0) from .env // tip: % enable debugging { debug: true }

    at _log (node_modules/dotenv/lib/main.js:131:11)

console.log
  ◊ injected env (0) from .env // tip: % suppress logs { quiet: true }

    at _log (node_modules/dotenv/lib/main.js:131:11)

PASS tests/api.test.js
  REST API Tests
    User Authentication Endpoints
      ✓ 1. POST /api/users/register - Should register user successfully (144 ms)
      ✓ 2. POST /api/users/register - Should reject duplicate email (12 ms)
      ✓ 3. POST /api/users/login - Should login user successfully and return JWT (94 ms)
      ✓ 4. POST /api/users/login - Should reject login with invalid password (97 ms)
    Post Endpoints
      ✓ 1. POST /api/posts - Should reject post creation without auth token (9 ms)
      ✓ 2. POST /api/posts - Should create post successfully with valid auth token (20 ms)
      ✓ 3. GET /api/posts - Should fetch all posts (16 ms)
      ✓ 4. GET /api/posts/:id - Should fetch single post by id (13 ms)
      ✓ 5. GET /api/posts/:id - Should return 404 for non-existent post (11 ms)
      ✓ 6. PUT /api/posts/:id - Should reject updates from non-owner user (12 ms)
      ✓ 7. PUT /api/posts/:id - Should update post successfully with owner token (20 ms)
      ✓ 8. DELETE /api/posts/:id - Should reject deletion from non-owner user (11 ms)
      ✓ 9. DELETE /api/posts/:id - Should delete post successfully with owner token (20 ms)

Test Suites: 1 passed, 1 total
Tests:       13 passed, 13 total
Snapshots:   0 total
Time:        1.507 s, estimated 4 s
Ran all test suites.
```